-- 以下为AI生成的图文笔记内容 --

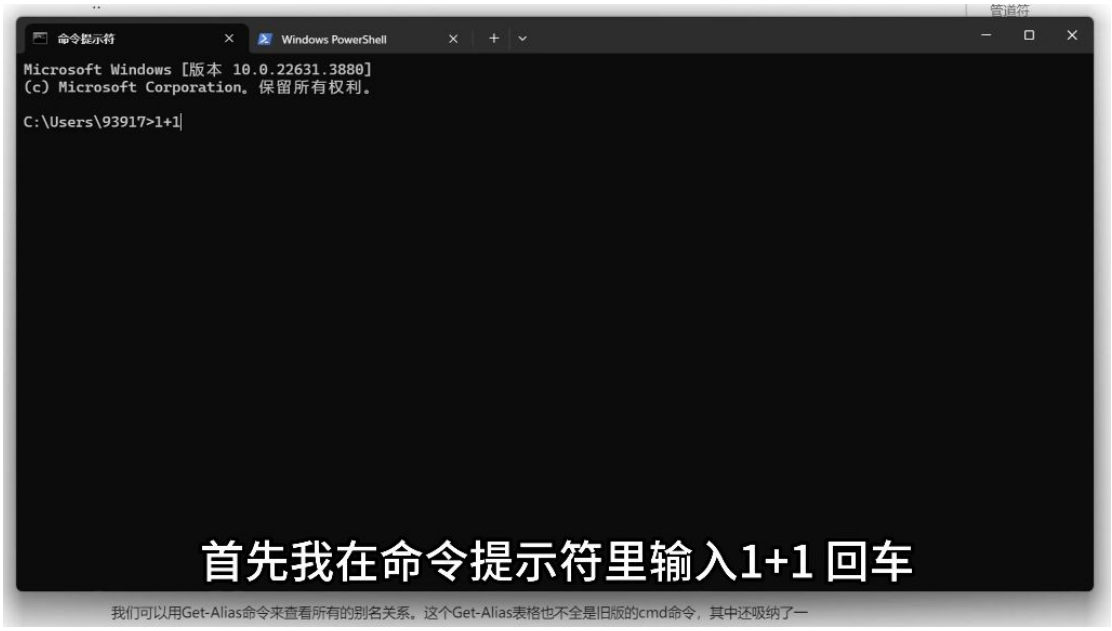
*一、命令行工具**00:07***

1. 命令行工具介绍 00:08

- **两种Shell类型**：
  - Command shell：最早内置的Windows shell，通过.bat文件自动化常规任务（如用户账户管理、夜间备份）
  - PowerShell：扩展Command shell功能，提供更强大的脚本语言环境
- **核心功能**：
  - 都支持人机交互和IT操作自动化
  - 脚本执行效率高于图形界面操作
  - 脚本可接受命令行所有可用命令

2. 命令提示符 vs Windows PowerShell 01:05

1）例题:命令提示符运算

- 

- 题目解析
  - Command shell无法识别"1+1"数学运算，直接报错
  - PowerShell能自动识别为数字运算，输出正确结果2
  - PowerShell支持变量定义和运算（如$y1=1），类似高级编程语言

2）作为CMD命令提示行的上位替代 01:34

- **本质区别**：
  - Command shell：继承自老旧DOS系统，功能有限
  - PowerShell：完整的脚本语言运行环境，支持：
    - 数学运算
    - 变量操作
    - 复杂数据处理
    - 面向对象编程
- 例题:get-process命令 01:51
  - **cmdlet特性**：
    - 命名规范：动词-名词格式（如Get-Process）

- 底层实现：基于.NET库编写
- 功能扩展：如Set-Location不仅可切换文件目录，还能操作注册表/证书存储目录

3）别名 02:47

-

- **设计目的**：兼容旧版cmd和Linux命令
- **实现方式**：
  - cmd命令映射为cmdlet（如cd→Set-Location）
  - 吸纳Linux命令（如ls→Get-ChildItem）
- **查看方法**：使用Get-Alias命令查看所有别名关系

4）应用案例 03:30

- 例题:ls命令使用

-

  - 题目解析
    - Command shell：不识别ls命令（非Windows原生命令）
    - PowerShell：通过别名机制支持ls命令，显示完整目录结构
    - 优势：降低Linux用户学习成本，实现命令通用性

5）命令过渡总结 04:00

- **兼容性优势**：
  - cmd用户：无缝过渡，无需学习新命令
  - Linux用户：直接使用熟悉命令（如pwd/ls/cp等）
- **推荐建议**：PowerShell完全可作为cmd的上位替代

6）管道符 04:19

- 例题:管道符命令拼接 04:37

-

  - **核心功能**：将上条命令输出作为下条命令输入
  - **优势**：
    - 支持.NET对象传递（非纯文本）
    - 保留数据结构，便于复杂数据处理
  - **典型案例**：
    - 获取CPU占用前5的进程：Get-Process | Sort-Object CPU -Descending | Select-Object -First 5
    - 计算目录总大小：Get-ChildItem | Measure-Object -Property Length -Sum

7）脚本语言 05:49

- 例题:if嵌套bat文件 06:07

C: > Users > 93917 > Desktop > 测试 > cmd.bat

```
@echo off
setlocal

set var1=1
set var2=2

if %var1%==1 goto FirstLevelMet
goto End

:FirstLevelMet
echo First level condition met
if %var2%==2 goto SecondLevelMet
goto End

:SecondLevelMet
echo Second level condition met

:End
pause
```

比如这个.bat脚本文件由于不能使用if嵌套

-
  - **bat文件缺陷**：
    - 扩展名：.bat
    - 不支持if嵌套，需用goto实现条件跳转
    - 代码可读性差，调试困难
- 例题:if嵌套powershell文件 06:26

cmd.bat | powershell.ps1

C: > Users > 93917 > Desktop > 测试 > powershell.ps1

```
$var1 = 1
$var2 = 2

if ($var1 -eq 1) {
    Write-Output "First level condition met"
    if ($var2 -eq 2) {
        Write-Output "Second level condition met"
    }
}
pause
```

可以使用括号与嵌套if表示代码层级

-
  - **ps1文件优势**：
    - 扩展名：.ps1

        - 支持现代编程语法（括号嵌套/代码块）
        - 示例代码行数减少50%，更符合人类阅读习惯

8）cmdlet补充 06:47

- 例题:get-command命令 06:53
    - **常用命令**：
        - Get-Command：查看所有支持命令
        - Update-Help：更新帮助文档
        - Get-Help：查看命令详细说明
- 例题:ls命令使用 07:21
- 例题:mkdir命令使用 07:39
- 例题:cp命令使用 07:57
- 例题:rm命令使用 08:09
- 例题:export-csv命令使用 08:24

打开看一下

-
    - **数据导出功能**：
        - Export-Csv：将进程信息导出为CSV格式

- ConvertTo-Html：生成HTML格式报告
- 优势：支持结构化数据输出，便于后续处理

*二、内容总结**08:59***

- **核心结论**：PowerShell是更现代、更强大的命令行环境
- **三大优势**：
  - 完整脚本语言支持
  - 全面兼容cmd和Linux命令
  - 面向对象的管道操作
- **推荐场景**：所有需要Windows自动化的场景都应优先使用PowerShell

三、知识小结

知识点	核心内容	易混淆点/对比维度	功能亮点
CMD与PowerShell的区别	CMD是Windows早期继承自DOS的命令行工具，功能有限，仅支持基础Windows命令；PowerShell是现代化扩展工具，支持脚本语言和.NET对象操作	**命令兼容性**：CMD仅支持Windows命令，PowerShell兼容CMD命令（通过别名）并扩展专属Cmdlet **输出类型**：CMD返回纯文本，PowerShell返回.NET对象 **脚本能力**：CMD批处理（.bat）语法老旧，PowerShell脚本（.ps1）支持现代编程结构（如嵌套if）	**别名机制**：PowerShell通过别名兼容CMD和Linux命令（如ls、cat） **管道操作**：支持对象级管道拼接（如Get-Process | Sort CPU -Descending | Select -First 5）
PowerShell核心功能	完整的脚本语言环境，支持变量定义、数学运	**Cmdlet语法**：遵循动词-名词格式（如Get-Process、	**跨平台命令**：内置类Linux命令（pwd、mkdir、

	算（如1+1输出2）、复杂数据处理	Set-Location） **目录切换**：Set-Location不仅切换文件目录，还可操作注册表/证书存储	cp） **数据导出**：支持CSV/HTML格式输出（如Export-Csv、ConvertTo-Html）
脚本开发对比	CMD批处理（.bat）限制多（如不支持if嵌套），需依赖goto；PowerShell脚本（.ps1）语法现代，可读性强	**调试复杂度**：CMD脚本逻辑碎片化，PowerShell代码结构清晰 **功能扩展**：PowerShell可直接调用.NET库	**内置帮助系统**：Get-Help查看命令文档，Update-Help更新帮助库